

# AI Prompt Injection — Incident Response Playbook

What the SOC does when an agent has already been manipulated

Prevention checklists tell you how to stop prompt injection. This playbook is for the moment prevention failed — an agent is confirmed or suspected to be acting on attacker-supplied instructions. **Agent incidents are not app incidents.** A compromised web server has a process you kill; a compromised agent has a *machine identity*, *persistent memory*, and *live tool access* to email, records, and money. Containment therefore means more than stopping a process — it means revoking what the agent can do and cleaning what it has remembered. This runbook follows the NIST 800-61 lifecycle, adapted for agents.

## Classify and set severity

Severity is a function of two things the assessment already scored: how sensitive the data in reach is, and how much the agent can act. An injected agent that can only chat is an incident; one that can send email or move money is a crisis.

| Severity       | Trigger                                                                                                     | Clock          |
|----------------|-------------------------------------------------------------------------------------------------------------|----------------|
| SEV-1 Critical | Injected agent has reachable high-impact tools (send / delete / pay / execute) or sensitive data in context | Immediate page |
| SEV-2 High     | Injection confirmed; data exposure possible but no destructive tool fired                                   | < 1 hour       |
| SEV-3 Medium   | Injection attempt detected; guardrails held, no action taken                                                | Same day       |
| SEV-4 Low      | Suspicious pattern / probing; unconfirmed                                                                   | Triage queue   |

### PHASE 1 — Detect & analyze

Confirm it is really injection, not a benign odd answer. The tell is **provenance**: instruction-shaped behavior traceable to content the agent read, not to the authorized user.

- Pull the full session: user prompt, retrieved / tool content, the agent's reasoning, and every tool call.
- Identify the injection source — input, memory, a retrieved document, a tool response, a web page, or a multimodal file.
- Establish blast radius: which tools were reachable, what data was in context, which identity the agent acted under.
- Decide direct vs indirect. Indirect (poisoned document / page) means the payload persists — others will hit it too.
- Confirm whether any action already fired (email sent, record changed, data returned).

### PHASE 2 — Contain

Stop the agent from acting **before** you finish the forensics. Agent containment is a set of revocations, not a single kill.

- Trigger the kill switch / disable the agent endpoint so no new sessions start.
- Revoke or rotate the agent's identity credential and tool tokens — assume they are acting as the attacker.
- Disable high-impact tool bindings (send, delete, pay, execute) at the gateway, not in the prompt.

- Quarantine poisoned state: the specific memory entries, thread, or vector-store documents carrying the payload.
- Isolate the affected session(s); preserve them as evidence before cleanup.
- If indirect, pull the poisoned document / page from the source so no other agent ingests it.

### PHASE 3 — Eradicate & recover

Remove the payload, close the surface it came through, and prove the agent is clean before it goes back.

- Purge poisoned memory / thread / vector entries; verify no residual instruction remains in context.
- Rotate all credentials the agent could have exposed; review anything the agent touched during the window.
- Patch the injection surface (add content isolation, output validation, or an approval gate on the abused tool).
- **Re-assess before restore** — run the agent back through assessment; do not return it to production on a No-Go.
- Restore from a known-good configuration; monitor closely on return.

### PHASE 4 — Post-incident

The incident is not closed until the detection that missed it exists.

- Write the timeline: source, blast radius, actions fired, dwell time, containment gaps.
- Convert the attack into a detection rule (below) and a red-team test case for the assessment.
- Update the agent's risk tier and the crosswalk finding; feed lessons back to the owning team.

## Detection ideas for the SIEM (illustrative — adapt to your log schema)

If your agent logging is right (see Playbook 04), these become hunts and scheduled rules. Tag each detection with its ATLAS technique so it slots into the same SOC workflow as your ATT&CK rules.

| Detection concept               | Signal to hunt                                                                                                                | ATLAS             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>Instruction-shaped input</b> | Override phrases / role-play framing in prompts or retrieved content from a source not trusted with that role                 | AML.T0051         |
| <b>Anomalous tool use</b>       | A tool firing outside the agent's behavioral baseline, or a high-impact tool right after new external content entered context | AML.T0086         |
| <b>Data egress via tool</b>     | Outbound send / export whose payload overlaps sensitive context the agent just read                                           | AML.T0086         |
| <b>Context / memory drift</b>   | Gradual cross-session behavior change with no config change — the indirect-poisoning signature                                | Context Poisoning |
| <b>Repeated refusals</b>        | A spike in guardrail triggers from one user / session — probing before a successful bypass                                    | AML.T0051         |

Illustrative Sentinel-style hunt (pseudo-KQL — not production):

```
AgentLogs
| where TimeGenerated > ago(24h)
| where ContextSource != "user" // content from a doc / tool / web
| where Prompt matches regex @"(?i)ignore (all|previous)|disregard|you are now"
| join kind=inner (AgentToolCalls
  | where ToolRisk in ("send","delete","pay","execute"))
  on SessionId
| project TimeGenerated, AgentId, ActingUser, ContextSource, ToolName, ToolParams
```

## Evidence to collect & host-side correlation

For attribution and any legal follow-up, preserve: the full session transcript with provenance tags, the poisoned artifact (document / page / file), agent identity and token usage, tool-call logs with parameters, and the config / prompt version in effect. When the agent executes tools on an endpoint, correlate with the **EDR** (e.g. CrowdStrike): the agent's tool call should have a matching host process, file, or network event — a tool action with no corresponding endpoint telemetry, or endpoint activity the agent did not log, is itself a finding.

---

### Framework mapping

Prompt-injection incidents map to **MITRE ATLAS AML.T0051 (LLM Prompt Injection)**, with **AML.T0086 (Exfiltration via AI Agent Tool Invocation)** when data leaves via a tool, plus the 2025 agent techniques (Context Poisoning, Memory Manipulation) for indirect cases; **OWASP LLM01**, and **LLM06 / Agentic ASI02** where a tool acted; **NIST AI RMF (Manage)**; and **EU AI Act Art.14 & Art.12** (oversight and logging). Tagging detections with these IDs is what lets an AI incident ride your existing SOC tooling.

**Safety.** This runbook is exercised with safe sample data in an authorized lab or staging environment only. Never use real credentials, tokens, or customer data, and never run adversarial probes against production. All test results are labeled as authorized testing.